



Lauda Local

Manual do usuário

Transcrição e análise de áudio e vídeo
100% no seu computador, sem nuvem e sem mensalidade.

Versão 0.15.0-beta | 24/09/2026

Windows, macOS e Linux

Sumário

1. O que é o Lauda Local	3
O que ele faz	3
O que ele não faz	3
2. Como abrir	4
Forma 1 — atalho na Área de Trabalho (a normal)	4
Forma 2 — a mesma janela, pelo terminal	4
Forma 3 — interface no navegador (alternativa)	4
Forma 4 — linha de comando (para repetir e automatizar)	4
A primeira execução é mais lenta	4
Conferindo se está tudo certo	4
3. A janela, campo a campo	5
As nove páginas da barra lateral	5
A página Novo trabalho, parte por parte	5
Os recursos do painel da direita	6
As páginas de resultado	7
Ollama: verificar e baixar pela janela	8
Quanto tempo ainda falta	8
Cobertura: o arquivo inteiro foi lido?	8
O diagnóstico da máquina	8
A página Desempenho: quanto da máquina ele pode usar	8
Se travar: o aplicativo volta do ponto salvo	9
Modo claro e modo escuro	10
4. Os arquivos gerados	11
Onde ficam	11
5. O relatório, seção por seção	12
Como ler o Bloco B	12
6. Todas as opções da linha de comando	13
Comandos	13
Opções de lauda run	13
7. Qual modelo escolher	15

Quanto tempo vai demorar?	15
Baixar modelos para uso offline	15
8. Diarização, resumo e camada visual	16
Identificando quem fala	16
Resumo automático com Ollama	16
Camada visual (só para vídeo)	16
9. Quando algo dá errado	17
O comando que responde quase tudo	17
10. Limites honestos e privacidade	18
O que esperar da transcrição	18
Privacidade	18
Licenças	18
Onde ficam as coisas	18

1. O que é o Lauda Local

Você aponta um arquivo de áudio ou vídeo. O aplicativo devolve um **laudo em texto** com tudo o que dá para extrair localmente: metadados técnicos do arquivo, diagnóstico da qualidade do áudio, idioma detectado, a transcrição completa com marcação de tempo e, quando você pedir, quem falou cada trecho.

Nenhum byte do seu arquivo sai da máquina. Os modelos de inteligência artificial são baixados uma única vez e, a partir daí, funcionam offline para sempre. Não há API paga, conta, cota nem envio para servidor nenhum.

O que ele faz

- Lê os metadados completos do arquivo (formato, codecs, resolução, tags, capítulos).
- Extrai e normaliza o áudio automaticamente, mesmo que a entrada seja vídeo.
- Avalia a qualidade: volume médio, picos, saturação, quanto é silêncio, quanto é fala.
- Transcreve offline com o Whisper, com filtro de voz para reduzir invenção de texto.
- Detecta o idioma sozinho, ou usa o que você mandar.
- Marca o tempo de cada trecho e, opcionalmente, de cada palavra.
- Identifica quem fala (SPEAKER_00, SPEAKER_01...) quando você liga a diarização.
- Gera a legenda .srt de todo trabalho, e o .vtt se você pedir.
- Escolhe sozinho GPU ou CPU e reduz o modelo se faltar memória.

O que ele não faz

- Não é editor: não corta, não converte nem exporta mídia.
- Não traduz.
- Não envia nada para a nuvem, nem para "melhorar o modelo".
- Não substitui revisão humana: o Whisper erra com sotaque forte, ruído, música e falas sobrepostas.

A regra de ouro do relatório

Se alguma etapa não puder rodar, o bloco correspondente não some do laudo: ele aparece como **[INDISPONÍVEL] Bloco: motivo**. Você sempre sabe o que faltou e por quê.

2. Como abrir

Há quatro formas de usar. Se você só quer transcrever um arquivo, use a primeira e pule o resto deste capítulo.

Forma 1 — atalho na Área de Trabalho (a normal)

Dê dois cliques no atalho **Lauda Local**. Abre uma **janela própria do programa**: não é site, não abre navegador, e não fica nenhuma janela preta de console atrás. Para encerrar, feche a janela.

Se o atalho ainda não existir, crie-o uma única vez, dentro da pasta do projeto:

```
powershell -ExecutionPolicy Bypass -File scripts\create_shortcut.ps1
```

Forma 2 — a mesma janela, pelo terminal

```
cd C:\Users\PC\Desktop\projetos\mediainatel-local  
.\.env\Scripts\lauda.exe app
```

Forma 3 — interface no navegador (alternativa)

Existe também uma interface que roda no navegador, útil quando você quer acessar o aplicativo de outro computador da mesma rede. Ela exige o Gradio instalado e sobe um servidor local em 127.0.0.1:

```
.\.env\Scripts\lauda.exe ui
```

Forma 4 — linha de comando (para repetir e automatizar)

É a forma mais rápida quando você já sabe o que quer, e a única que dá para colocar em um script:

```
.\.env\Scripts\lauda.exe run "C:\videos\entrevista.mp4" -m small -l pt --diarize -o .\saida
```

Atenção ao encadear comandos no Windows

O PowerShell que vem no Windows (versão 5.1) **não aceita &&** — ele responde “O token '&&' não é um separador de instruções válido nesta versão”. Use ; para separar comandos.

A primeira execução é mais lenta

Na primeira vez que você usa um modelo, ele é baixado (de 75 MB a 1,5 GB, conforme o tamanho) e guardado na pasta **models**. Da segunda vez em diante, tudo roda offline e a espera some. O mesmo vale para o modelo de identificação de falantes.

Conferindo se está tudo certo

O comando abaixo faz um diagnóstico completo do ambiente: ffmpeg, placa de vídeo, memória, modelos já baixados, diarização e resumo local.

```
.\.env\Scripts\lauda.exe doctor
```

3. A janela, campo a campo

A janela tem uma **barra lateral** com nove páginas. A primeira, **Novo trabalho**, é onde tudo acontece; as outras mostram o resultado e as preferências. Nada é enviado para lugar nenhum ao clicar em Processar — todo o trabalho acontece na sua máquina.

As nove páginas da barra lateral

Página	Para que serve
Novo trabalho	Escolher o arquivo, ligar os recursos e processar.
Relatório	O laudo completo, com uma aba por arquivo processado.
Registro	O que o programa está fazendo agora, linha a linha.
Transcrição	O texto corrido, com uma aba por arquivo transcrito.
Legendas	As legendas geradas, com os tempos de entrada e saída.
Arquivos	O que foi gerado agora e o histórico de todos os trabalhos.
Desempenho	Quanto da máquina o aplicativo pode usar.
Ajuda	Este passo a passo, sempre à mão.
Configurações	Tema, resumo das opções guardadas e “Restaurar padrões”.

A página Novo trabalho, parte por parte

Onde	O que fazer	Detalhes
Área pontilhada	Arraste o áudio ou vídeo para dentro dela, ou clique para abrir o explorador de arquivos do sistema.	Aceita qualquer áudio ou vídeo que o ffmpeg leia: mp4, mkv, mov, avi, webm, mp3, wav, m4a, flac, ogg, opus e outros. Escolhido o arquivo, aparece embaixo uma linha com miniatura, nome, tamanho e duração — e um X para trocar.
Painel da direita	Idioma, qualidade e as chavinhas dos recursos.	Pode deixar tudo como está. O único ajuste que quase sempre compensa é informar o idioma em vez de deixar no automático.
Pasta de saída	No pé do painel da direita. O botão Alterar abre o explorador.	É onde o .txt vai ficar. Já vem preenchida com a pasta saída do projeto; você também pode digitar o caminho direto no campo.
Vários de uma vez	Solte (ou marque no explorador) quantos arquivos quiser.	O primeiro vai para o cartão e o resto entra numa fila , mostrada logo abaixo. Eles rodam um por vez : dois modelos carregados ao mesmo tempo brigam pela mesma memória e travam a máquina. Pode soltar mais arquivos com o trabalho em andamento — vão para o fim da fila. O botão Esvaziar a fila descarta quem ainda não começou, sem interromper o que está rodando. A fila inteira roda com as opções de quando você clicou em Processar: mexer num interruptor no meio não muda os arquivos que ainda estão esperando.

Onde	O que fazer	Detalhes
Pausar	Com o trabalho em andamento, aparece ao lado de Processar.	Congela o processamento onde está: o uso de processador cai a zero e o trabalho volta do mesmo ponto quando você clicar em Retomar . A memória continua ocupada — pausar devolve o processador, não a RAM. Fechar o aplicativo pausado perde a etapa em andamento, mas não o que já foi transcrito.
Rodapé	O botão Processar e a trilha de cinco etapas.	A trilha acende conforme o trabalho anda: Arquivo, Pré-processamento, Transcrição, Pós-processamento e Conclusão. O nome exato da etapa e a porcentagem aparecem embaixo e no pé da barra lateral.

O que você escolhe fica guardado

Idioma, qualidade, os interruptores de recurso e a pasta de saída voltam como você deixou na última vez — ficam em `~/.lauda/ui.json`, junto do tema e dos limites de máquina. A página Configurações mostra o que está guardado e tem o botão **Restaurar padrões** para zerar tudo de uma vez.

Os recursos do painel da direita

Campo	O que faz	Quando mexer
Idioma	Deixe em Detectar automaticamente ou force um idioma.	Force sempre que souber qual é. Evita o erro clássico de o modelo escolher o idioma errado em áudio curto ou com ruído.
Qualidade	Tamanho do modelo de transcrição, em linguagem simples. Maior significa mais preciso e mais lento.	Deixe em Recomendado para o dia a dia. Suba em material importante.
Diarização de falantes	Cada trecho passa a vir com um rótulo (SPEAKER_00, SPEAKER_01...) e o relatório ganha o tempo de fala de cada pessoa.	Entrevistas, reuniões, podcasts.
Marcar o tempo das palavras	Acrescenta ao laudo o Bloco C, com início, fim e confiança de cada palavra.	Edição fina de legenda. Deixa o arquivo bem maior.
Gerar legenda .srt	Já vem ligado. A legenda é o Bloco B do laudo noutro formato, e sai junto com ele.	Deixe ligado. Desligar só economiza algumas dezenas de KB.
Gerar legenda .vtt	A mesma legenda no formato que o vídeo em página web usa.	Quando for publicar o vídeo num site.
Tamanho das legendas	Curtas (1 a 2 s), Equilibrada (5 a 8 s, o padrão) ou Blocos longos (até 1 min). Vale para .srt e .vtt.	Curtas para edição fina e karaokê — ficam bem melhores com “marcar o tempo das palavras” ligado. Blocos longos quando você quer o texto corrido dentro do player, não legenda de verdade.
Extraí miniaturas	Só para vídeo: conta as trocas de cena e salva uma miniatura a cada 30 segundos numa pasta ao lado do relatório.	Para ter noção do ritmo de edição. Custa uma varredura extra do vídeo.

Campo	O que faz	Quando mexer
Integrar com Ollama	Se o Ollama estiver rodando na máquina, acrescenta resumo, tópicos, itens de ação e citações.	Só faz efeito se você já usa o Ollama. Sem ele, o bloco fica marcado como indisponível e nada quebra.

As páginas de resultado

Relatório, **Transcrição** e **Legendas** são a mesma página com outro arquivo: uma faixa de abas no alto, uma para cada arquivo daquele tipo **que existe na pasta de saída** — inclusive os de outro dia. Clicar na aba troca o conteúdo; acima dele ficam o nome do arquivo de origem, a duração, o modelo e o caminho completo. A lista se atualiza ao abrir a página e ao terminar um trabalho, e o botão **Atualizar** serve para quando você mexer na pasta por fora.

Embaixo do texto, os mesmos três botões em todas: **Abrir o local do arquivo** (a pasta abre com o arquivo já selecionado, pronto para copiar), **Copiar o texto** e um terceiro que abre o laudo daquele trabalho — na própria página Relatório, ele abre o laudo que está na tela.

Na página **Legendas** há um botão a mais: **Gerar as que faltam**. Ele escreve a legenda dos trabalhos que você já processou, lendo os trechos do arquivo .data.json de cada um — os mesmos que o laudo imprime no Bloco B. Nada é transcrito de novo, e leva menos de um segundo para uma pasta inteira. Serve para o que foi processado antes de a legenda sair por padrão, e para trocar o tamanho das legendas sem refazer o trabalho: apague a legenda antiga, escolha o tamanho novo e clique. O que já existe nunca é sobrescrito.

Registro é página separada, e mostra o que está acontecendo agora: cada etapa aparece na hora em que acontece. Os dois botões dela abrem a pasta dos logs e copiam o registro — é o que eu peço quando alguém relata um problema.

- **Relatório** — o laudo completo de cada arquivo processado.
- **Registro** — o que está acontecendo agora, etapa por etapa.
- **Transcrição** — só o texto corrido, para ler ou copiar.
- **Legendas** — os arquivos .srt e .vtt, com os tempos como estão no arquivo, e o botão que cria as que faltam.
- **Arquivos** — em cima, a lista dos arquivos salvos e o resumo do processamento (tempo, velocidade, modelo, idioma, palavras, cobertura) com os avisos; embaixo, o **histórico** de todos os trabalhos já feitos.

O histórico tem uma linha por trabalho, do mais recente para o mais antigo, com quando, arquivo, duração, qualidade, onde rodou (CPU ou GPU), velocidade, cobertura, buracos, avisos, se teve resumo do Ollama e a **confiança** da transcrição. No fim vem “O que merece um olhar”: os trabalhos que falharam, os que tiveram bloco indisponível e os que cobriram menos de 95% do arquivo. Os dois botões do rodapé abrem o laudo mais recente e limpam a lista — limpar apaga só a lista, nunca os arquivos já gerados.

A confiança em uma palavra

Vem do **avg_logprob** que o próprio modelo informa: quanto ele “acreditou” no que escreveu. O número sozinho engana (-0,28 parece ruim e é ótimo), então a coluna mostra primeiro a palavra — **alta**, **média** ou **baixa** — e o número depois. Confiança baixa costuma ser áudio ruim, idioma errado ou modelo pequeno demais, nessa ordem.

Ollama: verificar e baixar pela janela

A página Configurações mostra se o servidor do Ollama está no ar e se o modelo está baixado. Se não estiver, o botão **Baixar** faz o download ali mesmo, com a porcentagem e os gigabytes na tela — são cerca de 9 GB, e uma barra parada por vinte minutos seria indistinguível de travamento.

Quanto tempo ainda falta

A partir de 8% do trabalho, o aplicativo passa a estimar o que falta pelo ritmo já medido — e mostra isso no rodapé e embaixo da etapa atual. Antes desse ponto ele não arrisca: os primeiros segundos são gastos carregando o modelo, e uma conta feita ali diria algo como “faltam 3 horas” para um vídeo de cinco minutos.

Cobertura: o arquivo inteiro foi lido?

Todo relatório traz, na seção 5, quanto da duração do arquivo virou texto. É a única forma de saber que nada foi pulado: um relatório com texto correto pode, ainda assim, ter deixado dez minutos para trás — e não há como desconfiar só lendo o que está lá.

Trecho sem texto que é **silêncio** não conta como buraco: não há o que transcrever numa pausa. O aplicativo já mede onde estão os silêncios durante o diagnóstico de áudio e usa essa medida para separar as duas coisas. Sobra listado só o que tem som e mesmo assim não virou texto — com o horário de início e fim, para você conferir no vídeo.

Abaixo de 95% ele avisa

A janela mostra quantos segundos ficaram de fora, e o aviso vai também para o log. Não é motivo para o trabalho falhar: vídeo com música longa ou trecho sem fala cai aí legitimamente. É informação para você decidir se vale reprocessar.

O diagnóstico da máquina

Ao abrir, o aplicativo mede cinco coisas e dá um veredito: **processador** (número de threads), **memória**, **placa de vídeo**, **espaço livre em disco** na pasta dos modelos e a presença do **ffmpeg**. O resultado fica na aba **Diagnóstico** da página Desempenho, item a item, sempre visível.

Veredito	O que significa	O que acontece
Boa / Dá conta	Nada preocupa.	Nada. O aviso não aparece.
Vai funcionar, mas devagar	Algum item está no limite — poucos núcleos, pouca RAM ou pouco disco.	Nada interrompe; o diagnóstico registra o motivo.
Não vai dar conta	Algum item está abaixo do mínimo, ou falta o ffmpeg.	Abre um aviso com o que foi medido e a escolha entre fechar o aplicativo e continuar mesmo assim . Dá para marcar “não avisar de novo neste computador”.

O aviso não é uma trava: nada quebra o computador. O que ele evita é você descobrir depois de meia hora de espera que faltava memória — e o texto aponta os controles desta mesma página como a saída para tornar o trabalho suportável.

A página Desempenho: quanto da máquina ele pode usar

A página tem três abas: **Resumo**, **Limites** e **Diagnóstico**.

Aba	Para que serve
Resumo	Quatro opções prontas — Leve, Equilibrado, Rápido e Máximo — e, embaixo, o que elas significam agora: onde a transcrição vai rodar, com quantas threads e com qual modelo. É a aba para quem não quer pensar em porcentagem.
Limites	Os quatro controles deslizantes, para quem quer afinar. Mexer em qualquer um apaga o destaque das opções prontas: passa a ser “ajuste manual”.
Diagnóstico	O que foi medido no seu computador, item a item, sempre na tela — não escondido atrás de um botão. É a aba para mandar junto quando algo der errado.

Ele avisa quando o limite não cabe no trabalho

O Resumo compara o que você pediu (qualidade, quem fala, Ollama) com o que os limites liberam. Se o trabalho pedir mais memória do que sobrou, aparece um aviso dizendo que o modelo será rebaixado — antes de você esperar meia hora para descobrir. E quando a transcrição roda na placa de vídeo, o Resumo lembra que o medidor do processador fica quase parado: isso é o esperado, não é o programa travado.

Os quatro controles da aba Limites decidem o quanto o aplicativo pode ocupar. Diminua se quiser continuar trabalhando no computador enquanto ele processa. A escolha fica salva para as próximas vezes, e o resumo abaixo dos controles mostra na hora o efeito: onde vai rodar e com qual modelo.

Controle	O que realmente faz
Processador (CPU)	Vira número de threads do motor e do ffmpeg. Em 25%, o aplicativo usa um quarto dos núcleos. Abaixo de 60% ele também passa a rodar em prioridade menor, para o computador continuar respondendo.
Memória (RAM)	Teto de memória considerado ao escolher o modelo quando o trabalho roda na CPU. Um modelo que não caiba nesse teto é trocado por um menor, e o relatório registra a troca.
Placa de vídeo (GPU)	Em 0% a placa é ignorada e tudo roda na CPU. Acima disso, controla o paralelismo do motor na placa.
Memória da placa (VRAM)	Teto de VRAM para escolher o modelo. É o que impede um modelo grande demais de estourar a placa.

Uma ressalva honesta sobre a GPU

O CUDA não tem um botão de “usar 50% da placa”. O controle mais próximo que dá para oferecer sem enganar é ligar/desligar e ajustar o paralelismo — e é isso que o controle faz. Já os limites de memória mudam de verdade qual modelo é carregado.

Se travar: o aplicativo volta do ponto salvo

Processar áudio longo demora, e coisas dão errado: a placa engasga, falta memória, o computador desliga. O aplicativo trata isso em duas camadas, sem você precisar fazer nada.

- **Pontos de retomada** — depois de cada etapa cara, o progresso é gravado no disco. Se o processamento morrer, rodar o mesmo arquivo com as mesmas opções continua de onde parou. Na prática: **a transcrição não é feita duas vezes**. O áudio já extraído também fica guardado.
- **Supervisor** — o trabalho roda num processo separado, vigiado. Se ele parar de dar sinal de vida, o supervisor encerra esse processo, volta do último ponto salvo e tenta de novo com o ambiente rebaixado: primeiro sem GPU, depois com um modelo menor.

Enquanto isso acontece, a janela avisa em amarelo o que está sendo feito, e a página **Arquivos** guarda o histórico da recuperação. Trocar a pasta de saída ou pedir legenda não invalida um ponto salvo; trocar o modelo, o idioma ou desligar a GPU, sim — nesses casos o resultado seria outro.

Modo claro e modo escuro

O botão na página **Configurações** alterna entre os dois. A escolha fica guardada no seu perfil e vale para as próximas vezes que você abrir o programa. **Sem nenhuma escolha salva, o aplicativo segue o tema do Windows** — quem já usa o sistema no escuro abre no escuro.

Para forçar um tema na abertura, sem mexer no que está salvo, use **--theme** com auto, claro ou escuro.

```
.\.venv\Scripts\lauda.exe app --theme escuro
```

A janela não trava enquanto processa

O trabalho pesado roda em segundo plano. Você pode mover a janela, trocar de página e ler o passo a passo enquanto a transcrição acontece.

4. Os arquivos gerados

Para um arquivo de entrada chamado **entrevista.mp4**, a pasta de saída recebe:

Arquivo	Conteúdo	Para quê
<code>entrevista.report.txt</code>	O laudo completo, em nove seções, pronto para ler ou imprimir.	É o produto principal. Abre em qualquer editor de texto.
<code>entrevista.transcript.txt</code>	Só o texto corrido, em parágrafos, sem cabeçalho nem marcação de tempo.	Para copiar e colar em um documento.
<code>entrevista.data.json</code>	Todos os dados estruturados: segmentos, palavras, falantes, métricas.	Para reaproveitar em outro programa ou script.
<code>entrevista.srt</code>	Legenda no formato SubRip, com no máximo duas linhas por trecho.	Players de vídeo, YouTube, editores.
<code>entrevista.vtt</code>	Legenda no formato WebVTT.	Players em navegador (HTML5).
<code>entrevista.assets/</code>	Pasta com as miniaturas extraídas do vídeo.	Só existe quando a camada visual está ligada.

Mesma entrada, mesma saída

Processar o mesmo arquivo com as mesmas opções gera um relatório com layout idêntico. Só mudam a data e os tempos medidos. Isso permite comparar duas execuções com qualquer ferramenta de diferença.

Onde ficam

Por padrão, na pasta **saida** dentro da pasta do aplicativo. Para abrir direto no Explorer:

```
explorer C:\Users\PC\Desktop\projetos\mediaintel-local\saida
```

5. O relatório, seção por seção

O laudo tem sempre as mesmas nove seções, sempre na mesma ordem. Isso é proposital: você aprende onde olhar uma vez e vale para todo arquivo.

Seção	O que traz	Para que serve na prática
1. Identidade do arquivo	Nome, caminho, tamanho, hash SHA-256, data de modificação e de processamento, versão do app, modelo, device.	Prova de integridade: o hash muda se o arquivo for alterado em um único byte.
2. Metadados técnicos	Container, duração, taxa de bits; para cada trilha de vídeo e de áudio: codec, resolução, quadros por segundo, canais, taxa de amostragem. Mais tags e capítulos.	Descobrir por que um arquivo está pesado, tremido ou com áudio ruim.
3. Qualidade e diagnóstico	Volume médio e de pico, saturação, quanto é silêncio, quanto é fala, tempo gasto em cada etapa e a lista de avisos.	Entender antes de reclamar do resultado: áudio baixo demais transcreve mal, e o relatório avisa.
4. Idioma	Idioma detectado, se foi automático ou imposto por você, e a confiança.	Confiança baixa costuma explicar transcrição estranha.
5. Transcrição completa	Bloco A: texto corrido em parágrafos. Bloco B: trechos com hora de início e fim. Bloco C (opcional): cada palavra com seu tempo.	O Bloco A é para ler; o B é para localizar no vídeo; o C é para edição fina de legenda.
6. Diarização	Quantas pessoas falaram, quanto tempo cada uma falou e em quantos trechos.	Reuniões e entrevistas: saber quem dominou a conversa.
7. Estrutura e resumo local	Contagem de palavras e caracteres, ritmo de fala, palavras mais frequentes sem as palavras vazias, e o resumo do Ollama quando disponível.	Uma leitura rápida do assunto antes de ler tudo.
8. Camada visual	Resolução real e exibida, número de cortes de cena e as miniaturas salvas.	Achar rapidamente o ritmo de edição de um vídeo.
9. Rodapé	Lista dos arquivos gerados, pasta de saída, pasta dos modelos e os erros parciais.	É onde você confere o que ficou de fora.

Como ler o Bloco B

```
[00:00:00.000 → 00:00:07.530] SPEAKER_00: Boa noite, obrigada por vir  
ao estúdio hoje.  
[00:00:07.530 → 00:00:14.580] SPEAKER_01: Boa noite. E um prazer estar  
aqui.
```

O formato do tempo é sempre **HH:MM:SS.mmm** (horas, minutos, segundos e milésimos). O rótulo do falante só aparece quando a diarização rodou.

6. Todas as opções da linha de comando

Tudo o que a interface oferece existe também como opção de comando — e algumas coisas só existem aqui.

Comandos

Comando	O que faz
<code>lauda run ARQUIVO</code>	Processa um arquivo e gera os relatórios.
<code>lauda app [--theme]</code>	Abre o aplicativo em janela própria. <code>--theme</code> aceita auto, claro ou escuro.
<code>lauda ui</code>	Abre a interface no navegador.
<code>lauda doctor</code>	Diagnóstico do ambiente: ffmpeg, GPU, memória, modelos, diarização, Ollama.
<code>lauda models</code>	Lista os modelos e diz quais cabem na memória da sua máquina.
<code>lauda checkpoints</code>	Lista os trabalhos interrompidos que dá para retomar. Com <code>--limpar</code> , apaga todos.

Opções de lauda run

Opção	Padrão	O que faz
<code>-o, --output</code>	<code>./saida</code>	Pasta onde gravar os resultados.
<code>-m, --model</code>	<code>small</code>	<code>tiny</code> , <code>base</code> , <code>small</code> , <code>medium</code> , <code>large-v3</code> , <code>large-v3-turbo</code> ou <code>distil-large-v3</code> .
<code>-l, --lang</code>	<code>auto</code>	<code>auto</code> , <code>pt</code> , <code>en</code> , <code>es</code> e outros códigos de idioma.
<code>-d, --device</code>	<code>auto</code>	<code>auto</code> , <code>cpu</code> ou <code>cuda</code> .
<code>--compute-type</code>	<code>auto</code>	Precisão numérica: <code>int8</code> , <code>int8_float16</code> , <code>float16</code> ou <code>float32</code> . Mexer aqui só se souber o que está fazendo.
<code>--beam-size</code>	<code>5</code>	Quantos caminhos o modelo considera. <code>1</code> é mais rápido e menos preciso.
<code>--vad / --no-vad</code>	<code>ligado</code>	Filtro de voz. Desligado, o modelo tende a inventar texto no silêncio.
<code>--words</code>	<code>desligado</code>	Gera os tempos por palavra (Bloco C).
<code>--diarize</code>	<code>desligado</code>	Liga a identificação de falantes.
<code>--diarize-backend</code>	<code>auto</code>	<code>auto</code> , <code>pyannote</code> ou <code>ecapa</code> .
<code>--num-speakers</code>	<code>-</code>	Número exato de pessoas falando. Melhora bastante o resultado quando você sabe.
<code>--min-speakers / --max-speakers</code>	<code>-</code>	Faixa esperada, quando você não sabe o número exato.
<code>--speaker-threshold</code>	<code>0.30</code>	Sensibilidade da separação de vozes no backend <code>ecapa</code> . Menor separa mais; maior junta mais.
<code>--srt / --no-srt</code>	ligado	Gera a legenda <code>.srt</code> .
<code>--vtt</code>	<code>desligado</code>	Gera também a legenda <code>.vtt</code> (vídeo na web).
<code>--no-json</code>	<code>-</code>	Não gera o arquivo de dados.

Opção	Padrão	O que faz
<code>--visual</code>	desligado	Cortes de cena e miniaturas (só vídeo).
<code>--summarize</code>	desligado	Bloco de resumo via Ollama local.
<code>--cpu</code>	100	Limite de CPU em % — vira número de threads.
<code>--ram</code>	50	Limite de RAM em % — teto para escolher o modelo.
<code>--gpu</code>	100	Limite de GPU em %. 0 desliga a placa.
<code>--vram</code>	100	Limite de VRAM em % — teto para escolher o modelo.
<code>--recover / --no-recover</code>	ligado	Supervisiona o processamento: encerra se travar e retoma do ponto salvo.
<code>--resume / --no-resume</code>	ligado	Usar os pontos de retomada salvos.
<code>--stall-timeout</code>	300	Segundos sem sinal antes de considerar travado.
<code>--attempts</code>	3	Quantas tentativas antes de desistir.
<code>--top-words</code>	25	Quantas palavras frequentes listar no relatório.
<code>--prompt</code>	-	Texto para ancorar o vocabulário: nomes próprios, siglas, jargão. Ex.: <code>--prompt "Fulano, ACME, CNPJ"</code> .
<code>--keep-temp</code>	desligado	Mantém o áudio temporário para depuração. Normalmente ele é apagado.
<code>-v / -q</code>	-	Log detalhado / apenas avisos e erros.

7. Qual modelo escolher

Esta é a decisão que mais afeta o resultado. A regra é simples: use o maior modelo que caiba na sua memória e cujo tempo de espera você tolere.

Modelo	Memória	Velocidade	Quando usar
<code>tiny</code>	0,5 GB	muito rápida	Só para testar se o arquivo abre. A qualidade é de rascunho.
<code>base</code>	0,7 GB	muito rápida	Pouco melhor que <code>tiny</code> . Raramente vale.
<code>small</code>	1,2 GB	rápida	O padrão. Bom equilíbrio para o dia a dia em português.
<code>medium</code>	2,4 GB	média	Quando <code>small</code> erra nomes e termos técnicos.
<code>large-v3-turbo</code>	2,2 GB	média	O melhor custo-benefício: quase a qualidade do <code>large</code> pela metade do tempo.
<code>distil-large-v3</code>	2,2 GB	média	Alternativa ao turbo.
<code>large-v3</code>	3,6 GB	lenta	Máxima qualidade. Não cabe em placas de 4 GB.

Você não consegue estourar a memória

Se o modelo pedido não couber, o aplicativo desce sozinho para o maior que couber e escreve o motivo no relatório. Se a placa de vídeo falhar no meio do caminho, ele refaz tudo no processador em vez de devolver um relatório vazio.

Quanto tempo vai demorar?

Depende demais de máquina e modelo para prometer número. Meça o seu caso: rode uma vez com **-m tiny** e olhe a linha **Velocidade** na seção 3 do relatório. Ela diz, por exemplo, **3,88x tempo real** — ou seja, um áudio de 10 minutos levou pouco menos de 3 minutos.

Baixar modelos para uso offline

Se você vai trabalhar sem internet, baixe antes o que for usar. Os arquivos vão para a pasta **models**:

```
..\venv\Scripts\python.exe scripts\download_models.py --model small --model large-v3-turbo
```

8. Diarização, resumo e camada visual

Identificando quem fala

A diarização separa as vozes e rotula cada trecho. São dois motores possíveis, e o aplicativo escolhe sozinho o que estiver disponível:

Motor	Precisa de cadastro?	Qualidade	Observação
ecapa	Não	Boa	É o padrão prático. Funciona sem conta, sem token e sem burocracia.
pyannote	Sim: conta e token no Hugging Face	Melhor	Lida melhor com falas sobrepostas. Exige aceitar os termos do modelo.

Se o resultado juntar duas pessoas em uma só, ou inventar gente demais, você tem dois ajustes. O primeiro é dizer o número de pessoas, quando souber:

```
.\.env\Scripts\lauda.exe run "reuniao.mp4" --diarize --num-speakers 3
```

O segundo é a sensibilidade. O valor padrão é 0,30: abaixe para separar mais vozes, aumente para juntar.

```
.\.env\Scripts\lauda.exe run "reuniao.mp4" --diarize --speaker-threshold 0.22
```

Limite honesto da diarização

Ela só rotula o que foi transcrito. Duas pessoas falando ao mesmo tempo viram um falante só, e vozes parecidas podem ser fundidas. É uma ajuda, não uma prova pericial.

Resumo automático com Ollama

Se você já usa o **Ollama** na máquina, o relatório ganha resumo, tópicos, itens de ação e citações — e continua tudo local, porque o Ollama roda no seu computador. Basta ter o servidor no ar e o modelo baixado:

```
ollama pull qwen3:14b
```

Depois use a opção **--summarize**, ou marque a caixa correspondente na interface. Transcrições longas são resumidas em duas passadas para caber na memória de modelos pequenos. Sem Ollama, o bloco simplesmente aparece como indisponível e o resto do relatório continua normal.

Camada visual (só para vídeo)

Com **--visual**, o aplicativo conta as trocas de cena e salva uma miniatura a cada 30 segundos numa pasta ao lado do relatório. Não há modelo de visão envolvido: é só ffmpeg, rápido e barato. Serve para ter uma noção do ritmo de edição e para localizar visualmente um trecho.

9. Quando algo dá errado

O aplicativo foi feito para nunca falhar em silêncio. Quase todo problema aparece escrito no relatório ou no terminal. Os mais comuns:

Sintoma	Causa provável e solução
O token '&&' não é um separador válido	Você usou && no PowerShell 5.1, que não aceita. Troque por ; ou rode um comando por linha. Não tem relação com o aplicativo.
Não encontrei ffmpeg	O ffmpeg não está instalado ou o terminal foi aberto antes da instalação. Instale com winget install --id Gyan.FFmpeg -e e abra um terminal novo.
Library cublas64_12.dll is not found	Faltam as bibliotecas CUDA. Rode pip install -r requirements-gpu.txt. Enquanto isso o aplicativo continua funcionando no processador.
A transcrição saiu vazia	O arquivo pode não ter fala. Confira na seção 3 do relatório se Fala detectada está como não; música e ruído são descartados de propósito.
Texto repetido ou inventado	Clássico do Whisper em silêncio ou volume muito baixo. Mantenha o VAD ligado (é o padrão), aumente o volume da gravação ou use modelo maior.
Nomes próprios e siglas errados	Use --prompt "Nome, SIGLA, jargão" para ancorar o vocabulário.
Diarização indisponível	Falta o pacote: pip install -r requirements-diarize.txt. Se aparecer erro de modelo gated, use --diarize-backend ecapa, que não pede token.
O processamento travou	O supervisor percebe sozinho, encerra e retoma do último ponto salvo, tentando de novo sem GPU e depois com um modelo menor. Se ainda assim não concluir, o ponto salvo continua no disco: rodar de novo aproveita o que já deu certo.
Ele está comendo a máquina inteira	Baixe os controles na página Desempenho. Abaixo de 60% de CPU o aplicativo também passa a rodar em prioridade menor.
Está muito lento	Confira na seção 3 qual device foi usado. Se apareceu cpu com GPU disponível, veja o erro de CUDA acima. Ou baixe o modelo.
Acentos quebrados no terminal	Console legado do Windows. Rode chcp 65001 antes, ou use o Windows Terminal.

O comando que responde quase tudo

```
.\.venv\Scripts\lauda.exe doctor
```

Ele mostra, em uma tela só: se o ffmpeg foi encontrado e onde, qual placa de vídeo e quanta memória existem, qual modelo o aplicativo escolheria sozinho, quais modelos já estão baixados, se a diarização está pronta e se o Ollama está no ar.

10. Limites honestos e privacidade

O que esperar da transcrição

- **Sotaque forte e fala rápida** derrubam a precisão, principalmente nos modelos menores.
- **Música e ruído de fundo** produzem texto inventado. O filtro de voz reduz o problema, mas não elimina.
- **Falas sobrepostas** viram uma linha só, e a diarização não consegue separar direito.
- **Silêncio prolongado** é a maior fonte de invenção: o modelo tenta preencher o vazio. Por isso o filtro de voz vem ligado.
- **Números, siglas e nomes próprios** erram com frequência. É onde a opção de prompt inicial mais ajuda.
- **Os tempos por palavra são estimativas** do próprio modelo, não medição acústica. Servem para legendar, não para perícia.

Revisão humana continua necessária

Para uso jurídico, jornalístico, médico ou acadêmico, trate a transcrição como um rascunho muito adiantado — nunca como documento final sem conferir.

Privacidade

O aplicativo não envia o seu arquivo para lugar nenhum. O único acesso à rede acontece no **download inicial dos modelos**, e depois disso nem isso é necessário — você pode inclusive bloquear o acesso à internet do processo, ou ligar o modo offline explícito, que impede qualquer download durante o processamento.

O servidor da interface escuta apenas em 127.0.0.1, ou seja, só a sua própria máquina consegue acessar. Ele nunca cria endereço público na internet.

Licenças

O código do aplicativo é MIT. As bibliotecas usadas são MIT, BSD ou Apache-2.0, e os pesos do Whisper são abertos, publicados pela OpenAI sob licença MIT. O ffmpeg não é redistribuído: o aplicativo apenas chama o que você instalou. O arquivo LICENSES.md, na pasta do projeto, traz a lista completa e as ressalvas de cada modelo.

Onde ficam as coisas

Pasta	Conteúdo
<code>saida\</code>	Os relatórios, transcrições, legendas e miniaturas.
<code>models\</code>	Os modelos baixados. Apagar libera espaço, mas força novo download.
<code>.venv\</code>	O ambiente Python do aplicativo. Não mexa.
<code>src\lauda\</code>	O código-fonte.
<code>docs\</code>	Este manual e o documento de desenho.
<code>tests\</code>	Os testes automatizados e as mídias de exemplo.